

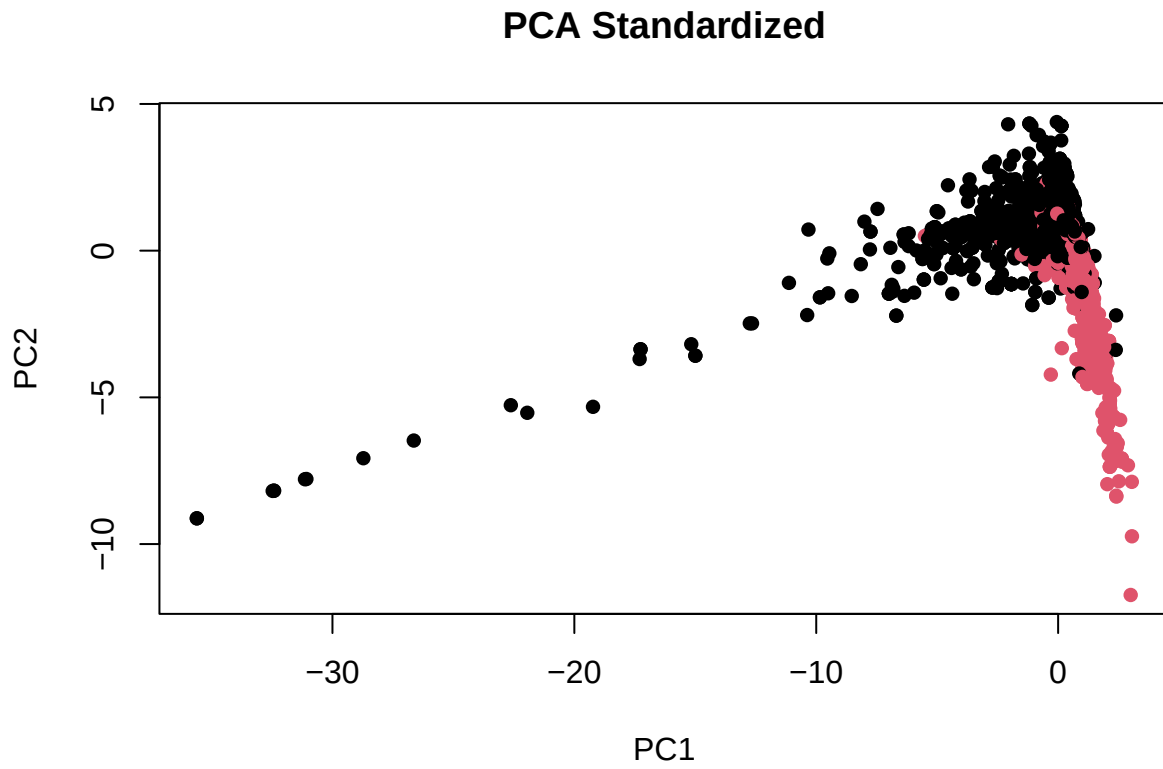
Final Project

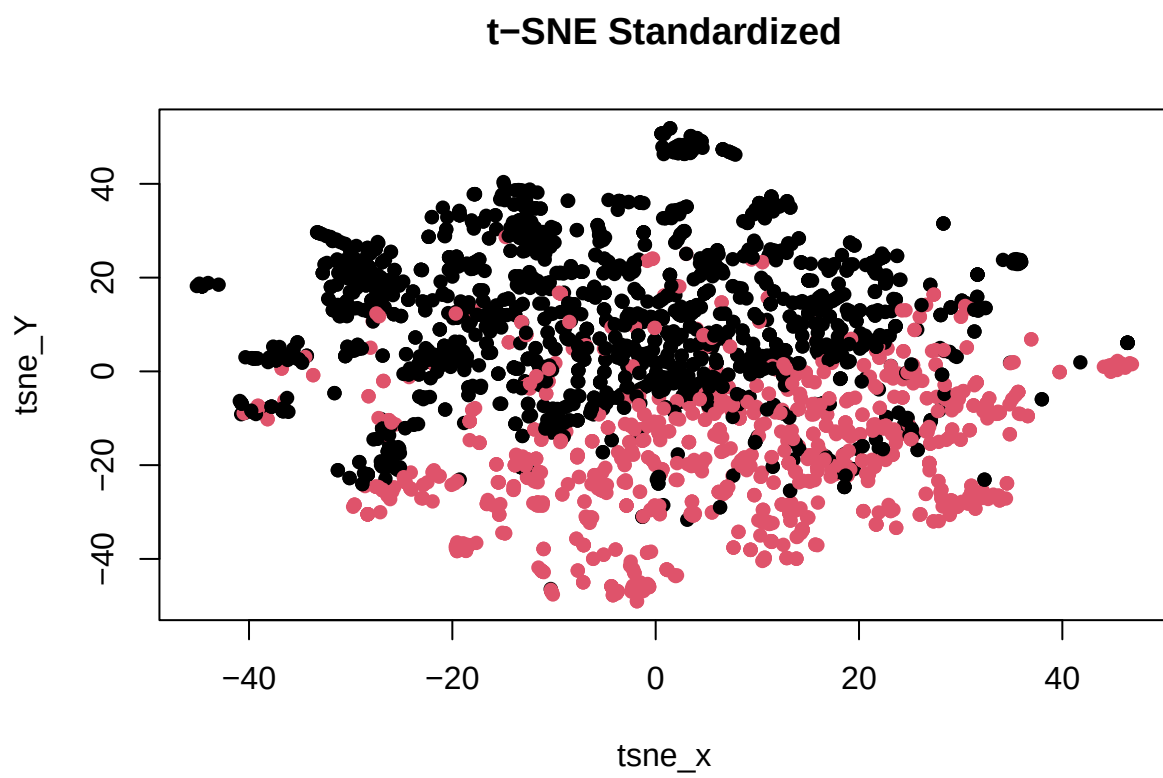
```
standardize <- function(x, mean, sd) {  
  return((x - mean) / sd)  
}  
  
train <- read.csv("spam-train.txt")  
test <- read.csv("spam-test.txt")  
colnames(train) <- c(paste0("V", 1:58))  
colnames(test) <- c(paste0("V", 1:58))  
  
train$V58 <- factor(train$V58, levels = c(0, 1))  
test$V58 <- factor(test$V58, levels = c(0, 1))  
  
train_std <- train  
test_std <- test  
train_std[, 1:57] <- scale(train_std[, 1:57])  
test_std[, 1:57] <- scale(test_std[, 1:57])  
  
train_discr <- as.data.frame(lapply(train[, 1:57], function(x) as.integer(x > 0)))  
test_discr <- as.data.frame(lapply(test[, 1:57], function(x) as.integer(x > 0)))  
train_discr$V58 <- train$V58  
test_discr$V58 <- test$V58  
  
train_log <- train  
test_log <- test  
train_log[, 1:57] <- log(train_log[, 1:57] + 1)  
test_log[, 1:57] <- log(test_log[, 1:57] + 1)
```

Visualization

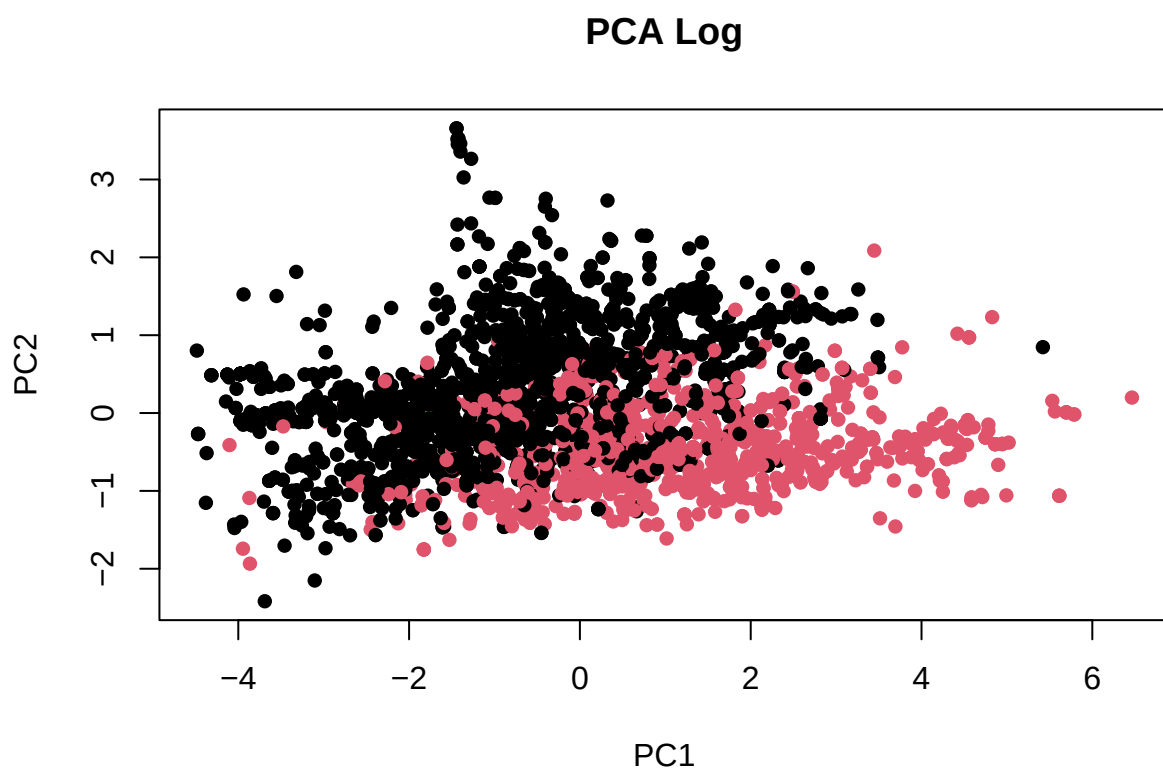
```
library(Rtsne)  
  
pca_scatter <- function(train, test, name) {  
  pca_res <- prcomp(train[, 1:57], center = TRUE, scale. = FALSE, retx = TRUE, rank. = 5)  
  
  scores <- predict(pca_res)  
  plot(scores[,1], scores[,2], col = train$V58, pch = 16,  
        xlab = "PC1", ylab = "PC2", main = paste("PCA", name))  
  
  tsne_results <- Rtsne(train[, 1:57], dims = 2, perplexity = 30, check_duplicates = FALSE)  
  
  train_data_tsne <- cbind(as.data.frame(tsne_results$Y), Spam = train$V58)  
  colnames(train_data_tsne) <- c("tsne_x", "tsne_y", "V58")  
}
```

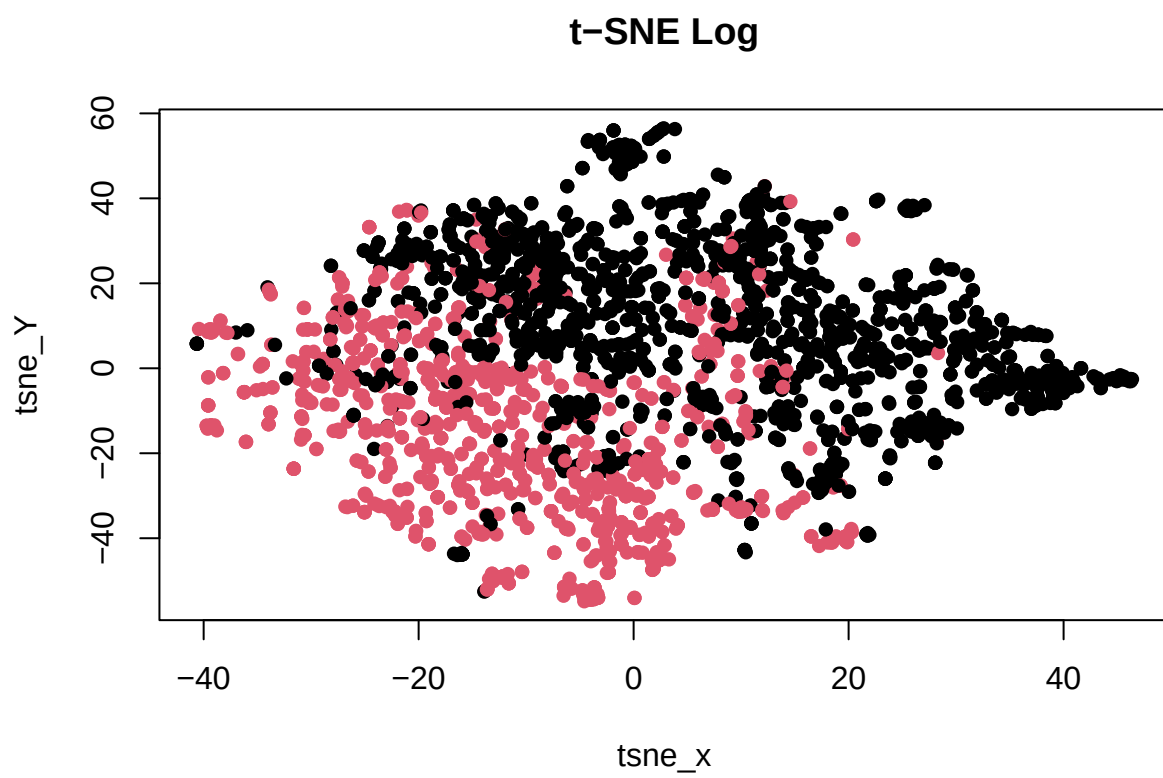
```
plot(train_data_tsne$tsne_x, train_data_tsne$tsne_y, col = train_data_tsne$V58, pch = 16,  
      xlab = "tsne_x", ylab = "tsne_Y", main = paste("t-SNE", name))  
}  
  
pca_scatter(train_std, test_std, "Standardized")
```





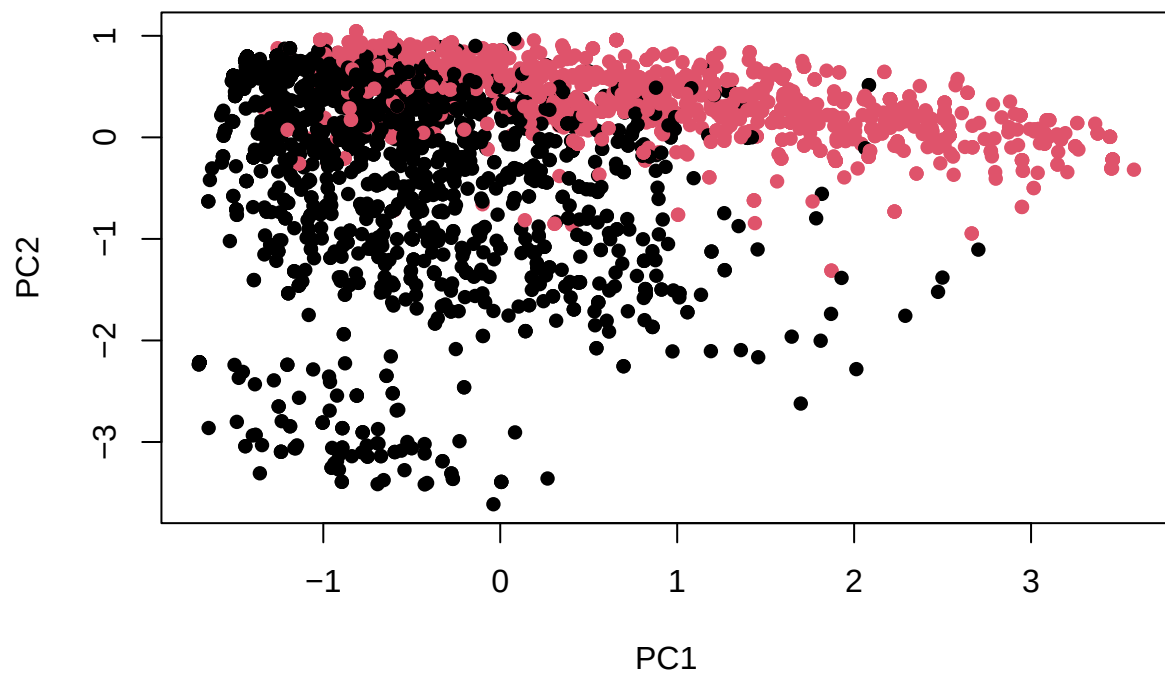
```
pca_scatter(train_log, test_log, "Log")
```

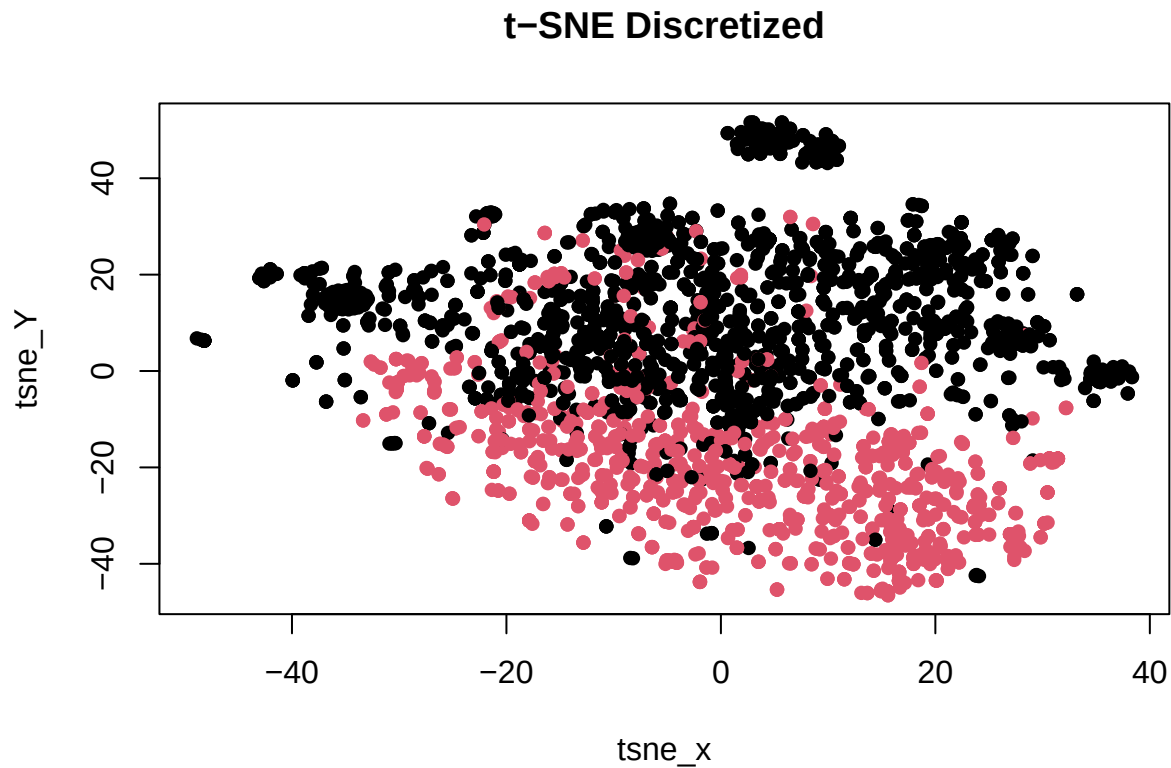




```
pca_scatter(train_discr, test_discr, "Discretized")
```

PCA Discretized





Logistic Regression

```
logistic_regression <- function(train, test, name) {
  model_logistic <- glm(V58 ~ ., family = "binomial", data = train)
  summary(model_logistic)

  pred_train_logistic <- ifelse(predict(model_logistic, type = "response") >= 0.5, 1, 0)
  pred_test_logistic <- ifelse(predict(model_logistic, newdata = test, type = "response") >= 0.5, 1, 0)

  train_error_logistic <- mean(pred_train_logistic != train$V58)
  test_error_logistic <- mean(pred_test_logistic != test$V58)

  cat("\n", name, "Logistic Regression\n")
  cat("Training error:", train_error_logistic, "\n")
  cat("Test error:", test_error_logistic, "\n")

  p_values <- summary(model_logistic)$coefficients[, 4]
  sig_predictors <- names(p_values[p_values < 0.05])
  cat("Significant Predictors:\n")
  print(sig_predictors[sig_predictors != "(Intercept)"])
}

logistic_regression(train_std, test_std, "Standardized")
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred

##
## Standardized Logistic Regression
## Training error: 0.07175473
## Test error: 0.0704501
## Significant Predictors:
## [1] "V4" "V5" "V7" "V8" "V9" "V11" "V16" "V17" "V19" "V20" "V21" "V23"
## [13] "V25" "V27" "V42" "V44" "V45" "V46" "V47" "V49" "V52" "V53" "V55" "V56"
## [25] "V57"
```

```
logistic_regression(train_log, test_log, "Log")
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred

##
## Log Logistic Regression
## Training error: 0.05772994
## Test error: 0.05675147
## Significant Predictors:
## [1] "V5" "V7" "V8" "V11" "V13" "V16" "V17" "V20" "V21" "V23" "V24" "V25"
## [13] "V27" "V28" "V33" "V35" "V37" "V42" "V43" "V45" "V46" "V49" "V52" "V53"
## [25] "V57"
```

```
logistic_regression(train_discr, test_discr, "Discretized")
```

```
## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading
```

```
##
## Discretized Logistic Regression
## Training error: 0.05707763
## Test error: 0.08088715
## Significant Predictors:
## [1] "V5" "V7" "V8" "V10" "V11" "V13" "V14" "V15" "V16" "V17" "V18" "V20"
## [13] "V21" "V23" "V24" "V25" "V27" "V28" "V37" "V42" "V43" "V44" "V45" "V46"
## [25] "V48" "V52" "V53" "V54"
```

This significant predictors we found all had p-values of less than 0.05.

Linear & Quadratic Discriminant Analysis

```
library(MASS)

lda_qda <- function(train, test, name) {
  model_lda <- lda(V58 ~ ., data = train)
  pred_train_lda <- predict(model_lda)$class
  pred_test_lda <- predict(model_lda, newdata = test)$class
```

```

train_error_lda <- mean(pred_train_lda != train$V58)
test_error_lda <- mean(pred_test_lda != test$V58)

model_qda <- qda(V58 ~ ., data = train)

pred_train_qda <- predict(model_qda)$class
pred_test_qda <- predict(model_qda, newdata = test)$class

train_error_qda <- mean(pred_train_qda != train$V58)
test_error_qda <- mean(pred_test_qda != test$V58)

cat("\n", name, "\n")
cat("\nLDA\n")
cat("Training error:", train_error_lda, "\n")
cat("Test error:", test_error_lda, "\n")

cat("\nQDA\n")
cat("Training error:", train_error_qda, "\n")
cat("Test error:", test_error_qda, "\n")
}

lda_qda(train_std, test_std, "Standardized")

```

```

##
## Standardized
##
## LDA
## Training error: 0.1017613
## Test error: 0.1030659
##
## QDA
## Training error: 0.1787345
## Test error: 0.1748206

```

```
lda_qda(train_log, test_log, "Log")
```

```

##
## Log
##
## LDA
## Training error: 0.0603392
## Test error: 0.06523157
##
## QDA
## Training error: 0.1588389
## Test error: 0.1572081

```

```
lda_qda(train_discr[, -c(55, 56, 57)], test_discr, "Discretized")
```

```

##
## Discretized

```

```
##
## LDA
## Training error: 0.07077626
## Test error: 0.08414873
##
## QDA
## Training error: 0.1014351
## Test error: 0.09980431
```

Linear & Nonlinear SVM

```
library(e1071)

lin_nonlin_svm <- function(train, test, name) {
  model_svm_linear <- svm(V58 ~ ., data = train, kernel = "linear", type = "C-classification")
  pred_train_svm_linear <- predict(model_svm_linear)
  pred_test_svm_linear <- predict(model_svm_linear, newdata = test)

  train_error_svm_linear <- mean(pred_train_svm_linear != train$V58)
  test_error_svm_linear <- mean(pred_test_svm_linear != test$V58)

  model_svm_rbf <- svm(V58 ~ ., data = train, kernel = "radial", type = "C-classification")
  pred_train_svm_rbf <- predict(model_svm_rbf)
  pred_test_svm_rbf <- predict(model_svm_rbf, newdata = test)

  train_error_svm_rbf <- mean(pred_train_svm_rbf != train$V58)
  test_error_svm_rbf <- mean(pred_test_svm_rbf != test$V58)

  cat("\n", name, "\n")
  cat("Linear SVM:\n")
  cat("Training error:", train_error_svm_linear, "\n")
  cat("Test error:", test_error_svm_linear, "\n\n")

  cat("Nonlinear SVM (RBF kernel):\n")
  cat("Training error:", train_error_svm_rbf, "\n")
  cat("Test error:", test_error_svm_rbf, "\n")
}

lin_nonlin_svm(train_std, test_std, "Standardized")

##
## Standardized
## Linear SVM:
## Training error: 0.06490541
## Test error: 0.07175473
##
## Nonlinear SVM (RBF kernel):
## Training error: 0.05153294
## Test error: 0.06457926
```

```
lin_nonlin_svm(train_log, test_log, "Log")
```

```
##  
## Log  
## Linear SVM:  
## Training error: 0.0541422  
## Test error: 0.05153294  
##  
## Nonlinear SVM (RBF kernel):  
## Training error: 0.03881279  
## Test error: 0.04500978
```

```
lin_nonlin_svm(train_discr, test_discr, "Discretized")
```

```
## Warning in svm.default(x, y, scale = scale, ..., na.action = na.action):  
## Variable(s) 'V55' and 'V56' and 'V57' constant. Cannot scale data.  
  
## Warning in svm.default(x, y, scale = scale, ..., na.action = na.action):  
## Variable(s) 'V55' and 'V56' and 'V57' constant. Cannot scale data.
```

```
##  
## Discretized  
## Linear SVM:  
## Training error: 0.0603392  
## Test error: 0.07436399  
##  
## Nonlinear SVM (RBF kernel):  
## Training error: 0.06164384  
## Test error: 0.07566862
```

Tree-based classifiers

```
tree_based <- function(train, test, name) {  
  tree_model <- rpart(V58 ~ ., data = train, method = "class")  
  
  train_pred_tree <- predict(tree_model, train, type = "class")  
  test_pred_tree <- predict(tree_model, test, type = "class")  
  
  rf_model <- randomForest(V58 ~ ., data = train, mtry = 4, importance=TRUE, ntree = 10, method = "class")  
  
  train_pred_rf <- as.integer(as.character(predict(rf_model, newdata=train, type = "class")))  
  test_pred_rf <- as.integer(as.character(predict(rf_model, newdata=test, type = "class")))  
  
  model_bagging <- randomForest(V58 ~ ., data = train, mtry = ncol(train) - 1)  
  
  pred_train_bagging <- as.integer(as.character(predict(model_bagging, newdata = train, type = "class")))  
  pred_test_bagging <- as.integer(as.character(predict(model_bagging, newdata = test, type = "class")))  
  
  tree_train_error <- mean(train$V58 != train_pred_tree)
```

```

tree_test_error <- mean(test$V58 != test_pred_tree)

rf_train_error <- mean(train$V58 != train_pred_rf)
rf_test_error <- mean(test$V58 != test_pred_rf)

train_error_bagging <- mean(pred_train_bagging != train$V58)
test_error_bagging <- mean(pred_test_bagging != test$V58)

cat("\n", name, "\n")
cat("\nDecision Tree\n")
cat("Training error:", tree_train_error, "\n")
cat("Test error:", tree_test_error, "\n")

cat("\nRandom Forest\n")
cat("Training error:", rf_train_error, "\n")
cat("Test error:", rf_test_error, "\n")

cat("\nBagging classifier (randomForest):\n")
cat("Training error:", train_error_bagging, "\n")
cat("Test error:", test_error_bagging, "\n")
}

tree_based(train_std, test_std, "Standardized")

```

```

##
## Standardized
##
## Decision Tree
## Training error: 0.09262883
## Test error: 0.1004566
##
## Random Forest
## Training error: 0.007175473
## Test error: 0.04435747
##
## Bagging classifier (randomForest):
## Training error: 0
## Test error: 0.04892368

```

```

tree_based(train_log, test_log, "Log")

```

```

##
## Log
##
## Decision Tree
## Training error: 0.09262883
## Test error: 0.09980431
##
## Random Forest
## Training error: 0.012394
## Test error: 0.04435747
##

```

```
## Bagging classifier (randomForest):  
## Training error: 0  
## Test error: 0.03261579
```

```
tree_based(train_discr, test_discr, "Discretized")
```

```
##  
## Discretized  
##  
## Decision Tree  
## Training error: 0.09849967  
## Test error: 0.1004566  
##  
## Random Forest  
## Training error: 0.03489889  
## Test error: 0.06588389  
##  
## Bagging classifier (randomForest):  
## Training error: 0.006849315  
## Test error: 0.04696673
```

Smallest Test Error Rate

```
library(DiceKriging)  
library(mlr)
```

```
## Loading required package: ParamHelpers
```

```
## Warning message: 'mlr' is in 'maintenance-only' mode since July 2019.  
## Future development will only happen in 'mlr3'  
## (<https://mlr3.ml-org.com>). Due to the focus on 'mlr3' there might be  
## uncaught bugs meanwhile in {mlr} - please consider switching.
```

```
##  
## Attaching package: 'mlr'
```

```
## The following object is masked from 'package:e1071':  
##  
## impute
```

```
## The following object is masked from 'package:ROCR':  
##  
## performance
```

```
library(mlrMBO)
```

```
## Loading required package: smooof
```

```
## Loading required package: checkmate
```

```
##
## Attaching package: 'checkmate'

## The following object is masked from 'package:DiceKriging':
##
##      checkNames

library(rgenoud)

## ## rgenoud (Version 5.9-0.3, Build Date: 2022-04-19)
## ## See http://sekhon.berkeley.edu/rgenoud for additional documentation.
## ## Please cite software as:
## ## Walter Mebane, Jr. and Jasjeet S. Sekhon. 2011.
## ## 'Genetic Optimization Using Derivatives: The rgenoud package for R.'
## ## Journal of Statistical Software, 42(11): 1-26.
## ##

learner <- makeLearner("classif.randomForest", predict.type = "response")
task <- makeClassifTask(data = train_log, target = "V58")

param_set <- makeParamSet(
  makeIntegerParam("mtry", lower = 1, upper = ncol(train_log) - 1),
  makeIntegerParam("ntree", lower = 100, upper = 500),
  makeIntegerParam("nodesize", lower = 1, upper = 20)
)

configureMlr(show.learner.output = FALSE, show.info = FALSE, on.par.without.desc = "quiet")
control <- makeTuneControlMBO(budget = 100)
cv <- makeResampleDesc("CV", iters = 5)

tuning_result <- tuneParams(learner, task, cv, par.set = param_set, control = control)

## Warning in generateDesign(control$infill.opt.focussearch.points, ps.local, :
## generateDesign could only produce 486 points instead of 1000!

## Warning in generateDesign(control$infill.opt.focussearch.points, ps.local, :
## generateDesign could only produce 48 points instead of 1000!

## Warning in generateDesign(control$infill.opt.focussearch.points, ps.local, :
## generateDesign could only produce 12 points instead of 1000!

## Warning in generateDesign(control$infill.opt.focussearch.points, ps.local, :
## generateDesign could only produce 996 points instead of 1000!

## Warning in generateDesign(control$infill.opt.focussearch.points, ps.local, :
## generateDesign could only produce 102 points instead of 1000!

## Warning in generateDesign(control$infill.opt.focussearch.points, ps.local, :
## generateDesign could only produce 20 points instead of 1000!

## Warning in generateDesign(control$infill.opt.focussearch.points, ps.local, :
## generateDesign could only produce 825 points instead of 1000!
```

```
## Warning in (function (fn, nvars, max = FALSE, pop.size = 1000, max.generations =  
## 100, : Stopped because hard maximum generation limit was hit.
```

```
## Warning in generateDesign(control$infill.opt.focussearch.points, ps.local, :  
## generateDesign could only produce 406 points instead of 1000!
```

```
## Warning in generateDesign(control$infill.opt.focussearch.points, ps.local, :  
## generateDesign could only produce 150 points instead of 1000!
```

```
## Warning in generateDesign(control$infill.opt.focussearch.points, ps.local, :  
## generateDesign could only produce 462 points instead of 1000!
```

```
## Warning in generateDesign(control$infill.opt.focussearch.points, ps.local, :  
## generateDesign could only produce 170 points instead of 1000!
```

```
## Warning in generateDesign(control$infill.opt.focussearch.points, ps.local, :  
## generateDesign could only produce 336 points instead of 1000!
```

```
## Warning in generateDesign(control$infill.opt.focussearch.points, ps.local, :  
## generateDesign could only produce 130 points instead of 1000!
```

```
best_mtry <- as.integer(tuning_result$x$mtry)  
best_ntree <- as.integer(tuning_result$x$ntree)  
best_nodesize <- as.integer(tuning_result$x$nodesize)
```

```
final_rf_model <- randomForest(V58 ~., data = train_log, ntree = best_ntree, nodesize = best_nodesize, mtry = best_mtry)
```

```
pred_train_opt <- predict(final_rf_model, newdata = train_log)  
train_error_opt <- mean(pred_train_opt != train_log$V58)
```

```
pred_test_opt <- predict(final_rf_model, newdata = test_log)  
test_error_opt <- mean(pred_test_opt != test_log$V58)
```

```
cat("\nRandom Forest (Optimized):\n")
```

```
##  
## Random Forest (Optimized):
```

```
cat("Train error:", train_error_opt, "\n")
```

```
## Train error: 0.001304631
```

```
cat("Test error:", test_error_opt, "\n")
```

```
## Test error: 0.02544031
```

```
cat("\nOptimal Hyperparameters:\n")
```

```
##  
## Optimal Hyperparameters:
```

```
print(tuning_result$x)
```

```
## $mtry  
## [1] 8  
##  
## $ntree  
## [1] 246  
##  
## $nodesize  
## [1] 1
```

```
““
```

		Classification Method							
		Logistic Regr.	LDA	QDA	Linear SVM	Nonlinear SVM	Decision Tree	Random Forest	Bagging
Dataset	Standardized (train)	0.07175473	0.1017613	0.1787345	0.06490541	0.05153294	0.09262883	0.00717547 3	0
	Standardized (test)	0.0704501	0.1030659	0.1748206	0.07175473	0.06457926	0.1004566	0.04435747	0.04892368
	Log (train)	0.05772994	0.0603392	0.1588389	0.0541422	0.03881279	0.09262883	0.012394	0
	Log (test)	0.05675147	0.06523157	0.1572081	0.05153294	0.04500978	0.09980431	0.04435747	0.03261579
	Discretized (train)	0.05707763	0.07077626	0.1014351	0.0603392	0.06164384	0.09849967	0.03489889	0.00684931 5
	Discretized (test)	0.08088715	0.08414873	0.09980431	0.07436399	0.07566862	0.1004566	0.06588389	0.04696673

The table is colored with the worst performing (high error) runs in red and the best performing (low error) runs in green.

QDA and Decision Trees seemed to have the worst overall performance while Bagging and Random Forests had the best performance. The log transformed data also performed better than the discretized and standardized datasets for all methods except for QDA where discretized performed the best.

We chose to optimize the Random Forest classification using Bayesian Optimization to find optimal values for mtry, ntree, and nodesize. We found the optimal parameters to be mtry=8, ntree=246, and nodesize=1 after 100 iterations of optimization. This gave us a training error of 0.001304631 and a test error of 0.02544031 which is lower than the other results. Even with default parameters, random forests performed very well so it would be highly recommended.